

NP-Completeness

Decision Problems

Definition: Problems with YES/NO answer only.

- **Example:** Is there a Hamiltonian cycle in graph G ? (YES/NO)
- **Example:** Can we color graph G with k colors? (YES/NO)
- **Example:** Does Boolean formula F have a satisfying assignment? (YES/NO)
- **Related:** Optimization problems ask "what is minimum/maximum"

Class P: Polynomial Time

Definition: Problems **solvable** in polynomial time.

- **Time:** Algorithm produces solution in $O(n)$ steps
- **Example:** Sorting ($O(n \log n)$), MST ($O(E \log E)$)
- **Example:** Graph connectivity (DFS/BFS: $O(V+E)$)
- **Verification:** Check solution easily in polynomial time

Class NP: Nondeterministic Polynomial

Definition: Problems **verifiable** in polynomial time.

- Solution might be hard to find, but easy to verify
- **Example:** Hamiltonian cycle—guess path, verify in $O(n)$
- **Example:** 3-SAT—guess assignment, verify in $O(n)$
- **Key:** $P \subseteq NP$ (all solvable problems are verifiable)

P vs NP: The \$1,000,000 Question

Is $P = NP$? (Unsolved since 1971)

NP-Hard (NPH) Problems

Definition: All NP problems reduce to this problem in polynomial time.

- **Hardness:** As hard as the hardest NP problem
- **May not be in NP:** Might not even be verifiable in poly time
- **Example:** Traveling Salesman Problem (TSP)
- **Example:** Halting problem (not in NP at all)

NP-Complete (NPC) Problems

Definition: NP-Hard AND in NP. Hardest problems in NP.

- **Implication:** If any NPC problem is in P, then $P = NP$
- **Example:** SAT (Boolean Satisfiability)
- **Example:** Knapsack, Graph Coloring, Clique, Vertex Cover
- **Strategy:** Prove NPC by reduction from known NPC problem

Relationships: P, NP, NPC, NPH

Assuming $P \neq NP$ (most likely scenario)

Classic NP-Complete Problems

SAT

Boolean satisfiability

3-SAT

3 literals per clause

Clique

Largest complete subgraph

Vertex Cover

Min vertices covering edges

TSP

Shortest tour visiting all

Knapsack

Max value in capacity

How to Prove NP-Completeness

- **Step 1:** Show problem is in NP (verifiable in poly time)
- **Step 2:** Reduce known NPC problem to this problem in poly time
- **Implication:** If known NPC $\leq$ your problem, then your problem is NPC
- **Base Case:** SAT is NPC (Cook's Theorem, 1971)

Polynomial Time Reduction ($\leq$)

Problem A reduces to B if A's solution converts to B's solution in poly time.

- **Notation:** $A \leq B$ means "A reduces to B"
- **Implication:** If $B \in P$, then $A \in P$
- **Strategy:** If B is NPC and $A \leq B$, then A is NPC

Decision vs Optimization Problems

Decision

Can we find tour ≤ 1000 ?

YES/NO answer

Optimization

Find minimum cost tour

Numeric answer

If $P = NP$ (Apocalyptic Scenario!)

- ✗ **Cryptography breaks:** RSA, AES become solvable in poly time
- ✓ **Math proofs automated:** Verify $\Rightarrow$ Find proofs in poly time
- ✓ **Optimization solved:** TSP, Knapsack become tractable
- ✗ **Security compromised:** Digital signatures, blockchain vulnerable
- **Consensus:** Most believe $P \neq NP$ (no evidence for equality)

Practical Implications for Engineers

- **Recognize NPC problems:** Don't waste time finding optimal solutions
- **Use approximation algorithms:** Get near-optimal in poly time
- **Use heuristics:** Greedy, DP, local search for practical solutions
- **Branch & Bound:** For small instances, exact algorithms work
- **Accept non-optimal:** 95% of optimal is often good enough

Key Takeaways

- **P**: Solvable in polynomial time
- **NP**: Verifiable in polynomial time
- **NPC**: Hardest in NP, $P \subseteq NP \subseteq NPC$
- **NPH**: Harder than NPC, may not be in NP
- **Strategy**: Prove NPC via reduction from SAT or 3-SAT

Thank You!